

DAY 16

Pivot Table Transformation in PySpark

A pivot table transformation in PySpark is used to reshape data by converting distinct values from one column into multiple columns and aggregating corresponding values. It helps summarize large datasets and enables easier analysis of categorical information. In this project, PySpark pivot operations were used to transform raw healthcare data into a structured format suitable for reporting and analytics, improving data readability and supporting downstream business intelligence requirements.

Data Transformations in PySpark

Several PySpark transformations were applied to cleanse, validate, and standardize the data before analysis. These transformations included filtering invalid records, handling null values, removing duplicates, splitting and extracting fields, creating derived columns, aggregating metrics, and performing joins across multiple datasets. The transformation pipeline ensured data quality, consistency, and readiness for reporting and dashboard visualization.

Dashboard Development in Databricks

A Databricks dashboard was developed to present key insights generated from the transformed datasets. The dashboard provides an interactive view of business and operational metrics through charts, tables, and summary visualizations. By leveraging Databricks SQL and visualization capabilities, stakeholders can monitor trends, analyze performance indicators, and make data-driven decisions using real-time insights derived from the processed data.

DAY 17

Data Analysis Using Window Functions and Transformations

Business analytics was performed using PySpark transformation and window functions to derive meaningful insights from large datasets. Transformations such as filtering, aggregation, joins, and column derivations were applied to prepare and standardize the data. Window functions including ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), and LEAD() were used to perform advanced analytical calculations, identify trends, compare records across groups, and generate business metrics without collapsing the underlying data. This approach enabled efficient analysis of operational performance, customer behavior, and key business indicators, supporting data-driven decision-making.

Q1. Why can't `groupBy()` calculate previous month's sales?

`groupBy()` aggregates rows and collapses the data into groups, causing the individual row order and month sequence to be lost. To calculate the previous month's sales, we need access to neighboring rows while preserving row-level data, which is possible using window functions like `lag()`. Therefore, `groupBy()` cannot determine the previous month's value for each record.

Q2. What is the difference between `sum("sales_amount")` and `sum("sales_amount").over(...)`?

- `sum("sales_amount")`
This is an aggregation function usually used with `groupBy()`. It combines multiple rows into a single summary value for each group.
Example: `df.groupBy("city").agg(sum("sales_amount"))`
 - `sum("sales_amount").over(...)`
This is a window aggregation function. It calculates totals while retaining every original row.
Example: `sum("sales_amount").over(windowSpec)`
 - `groupBy()` → Aggregates and reduces rows.
 - `over()` → Performs calculations while preserving rows.
-

Q3. When would you use: `row_number()`, `rank()` and `dense_rank()` for monthly sales reports?

- `row_number()`
Use when each row must receive a unique sequential number regardless of ties.
Use case:
 - Selecting the top sales month per city.
 - Removing duplicates.
- `rank()`
Use when ties should receive the same rank and gaps are acceptable.
Use case:
 - Competition-style ranking.

- Identifying top-performing cities or months.
- `dense_rank()`
Use when ties should receive the same rank but no gaps should appear.
Use case:
 - Business reports requiring continuous rankings.
 - Top N months or cities without skipped positions.

DAY 18

SQL Queries in PySpark

PySpark allows SQL queries to be executed on DataFrames by first registering them as temporary views. This provides a familiar SQL interface while leveraging Spark's distributed processing capabilities.

Example:

```
df_orders.createOrReplaceTempView("orders")

spark.sql("""
SELECT city,
       SUM(sales_amount) AS total_sales
FROM orders
GROUP BY city
ORDER BY total_sales DESC
""").show()
```

In this example, the `orders` DataFrame is registered as a temporary view and then accessed using a SQL query through `spark.sql()`. The query calculates the total sales for each city and displays the results in descending order of sales. This approach combines the flexibility of SQL with the scalability and performance of PySpark.

DAY 19

Analytical Project Pipeline

An analytical project begins by understanding the business problem and defining the objectives to be achieved. The next step is to understand the available data, including its structure, sources, and relevance to the problem. The data is then validated to ensure

completeness and accuracy, followed by cleaning to handle missing values, duplicates, and inconsistencies. Once cleaned, the data is transformed into a suitable format for analysis through aggregations, feature creation, and other preprocessing techniques. The transformed data is analyzed using statistical methods and business metrics to identify patterns and trends. These findings are then converted into actionable insights that support business decision-making. Finally, the results are presented through reports, dashboards, or visualizations, enabling stakeholders to make informed decisions based on the analysis.

1. Understand the Business Problem



2. Understand the Data



3. Validate the Data



4. Clean the Data



5. Transform the Data



6. Analyze the Data



7. Generate Insights



8. Present Findings